LORAP: Lora using a Product Penalization

Contents

Lora training using Product Penalization Read first [**zhang2024riemannianpreconditionedlorafinetuning**] which proposes the same method. Then consider extensions with momentum+truncation for a more practical method. I also think they have incorporate Adam incorrectly. This would all be a practical extension.

Alternatively. applying this to the product of the query and key matrix in an attention layer would be something completely new.

1 Introduction

Suppose we are given a pre-trained model given by

A data set:	$\mathbf{x} \sim \mathcal{D}$
A model:	$f(\boldsymbol{W}, \mathbf{x}) \in \mathbb{R}$
A loss function:	$f(\boldsymbol{W}) := \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[f(\boldsymbol{W}, \mathbf{x})\right]$.
Pre-trained weights:	$\boldsymbol{W}^*$ such that $\nabla f(\boldsymbol{W}^*) = 0$

Table 1: Pre-trained model

Now we would like to make a small change to our weights such that the resulting weights have a good error on a new data set. For this we will introduce a perturbation $\boldsymbol{\Delta W} = \boldsymbol{LR} \in \mathbb{R}^{m\times n}$ where $\boldsymbol{L} \in \mathbb{R}^{m\times r}$ and $\boldsymbol{R} \in \mathbb{R}^{r\times n}$, with $r \ll \min\{n, m\}$. In words, $\boldsymbol{\Delta W} = \boldsymbol{LR}$ is low rank. We then introduce a new loss given by

$$F(\boldsymbol{\Delta W}) = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}'}\left[h(\boldsymbol{W}^* + \boldsymbol{\Delta W}, \mathbf{x})\right], \tag{1}$$

where $\mathcal{D}'$ is a new data set.

If we apply gradient to the two components $\boldsymbol{L}$ and $\boldsymbol{R}$ of the perturbation separately, we get

$$\boldsymbol{L}_{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r}}{\operatorname{argmin}} F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{L} - \boldsymbol{L}_t\rangle + \frac{1}{2\lambda}\|\boldsymbol{L} - \boldsymbol{L}_t\|^2, \tag{2}$$

$$\boldsymbol{R}_{t+1} = \underset{\boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{R} - \boldsymbol{R}_t\rangle + \frac{1}{2\lambda}\|\boldsymbol{R} - \boldsymbol{R}_t\|^2. \tag{3}$$

where $\|\cdot\|$ is a given matrix norm.

One way to interpret (2) and (3), is that we are approximating our loss by its local linearization around $\boldsymbol{L}_t\boldsymbol{R}_t$, and then penalizing solutions in $\boldsymbol{L}$ and $\boldsymbol{R}$ that are far from $\boldsymbol{L}_t$ and $\boldsymbol{R}_t$, respectively. Here we argue that this is not the best form of penalization. Instead, we should penalize $\boldsymbol{LR}$ when it is far from $\boldsymbol{L}_t\boldsymbol{R}_t$.

2 Using a product norm for Lora

Consider gradient descent over the perturbation

$$\boldsymbol{\Delta W}^{t+1} = \underset{\boldsymbol{\Delta W}}{\operatorname{argmin}} F(\boldsymbol{W}^t) + \left\langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W}^t), \boldsymbol{\Delta W} - \boldsymbol{\Delta W}^t\right\rangle + \frac{1}{2\lambda}\|\boldsymbol{\Delta W} - \boldsymbol{\Delta W}^t\|^2.$$

Substituting in $\boldsymbol{\Delta W} = \boldsymbol{LR}$ and $\boldsymbol{\Delta W}^t = \boldsymbol{L}_t\boldsymbol{R}_t$ gives

$$\boldsymbol{\Delta W}^{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r}, \boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{LR} - \boldsymbol{L}_t\boldsymbol{R}_t\rangle + \frac{1}{2\lambda}\|\boldsymbol{LR} - \boldsymbol{L}_t\boldsymbol{R}_t\|^2. \tag{4}$$

3 Full best r approach

Assuming $\|\cdot\| = \|\cdot\|_F$, by completing the squares, the above is equivalent, up to additive constants, to solving

$$\boldsymbol{\Delta W}^{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r},\boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} \|\boldsymbol{LR} - (\boldsymbol{L}_t\boldsymbol{R}_t - \lambda\nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t\boldsymbol{R}_t))\|_F^2. \tag{5}$$

The solution to the above is given by the best rank r approximation of $\boldsymbol{L}_t\boldsymbol{R}_t - \lambda\nabla F(\boldsymbol{L}_t\boldsymbol{R}_t)$. That is, we obtain the new low-rank matrix by taking a gradient descent update and then truncate. This yields the following ideal stepping scheme: let $\boldsymbol{\Delta W}^0$ be a the initial rank r matrix and let $\mathcal{T}_r$ denote the operation that computes the best rank r approximation, then

$$\boldsymbol{\Delta W}^t = \boldsymbol{L}_t\boldsymbol{R}_t, \quad \boldsymbol{G}^t := \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W}^t), \quad \boldsymbol{D}^{t+1} = \boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t, \quad \boldsymbol{\Delta W}^{t+1} = \mathcal{T}_r\left(\boldsymbol{D}^{t+1}\right). \tag{6}$$

Computing the optimal rank r approximation to $\boldsymbol{D}^{t+1}$ requires $O(mn\min\{m,n\})$ operations, which becomes prohibitively expensive for large m and n. Instead, we can resort to randomized techniques, such as the randomized SVD, subspace iteration, Krylov subspace method, and the generalized Nyström method [**rsvd**; **MM15**; **tropp2023randomized**; **nakatsukasa2020fast**]. These methods can be proven to return rank r approximations $\widehat{\boldsymbol{D}}^{t+1}$ satisfying

$$\|\boldsymbol{D}^{t+1} - \widehat{\boldsymbol{D}}^{t+1}\|_F \leq (1+\varepsilon)\|\boldsymbol{D}^{t+1} - \mathcal{T}_r(\boldsymbol{D}^{t+1})\|_F,$$

with high probability, for some ε depending on the method. In other words, these methods compute cheap and *near-optimal* low-rank approximations, sacrificing some accuracy for speed.

3.1 Subspace iteration and the randomized SVD

Given a random matrix $\boldsymbol{\Omega} \in \mathbb{R}^{n\times r}$, the basic form of subspace iteration proceeds by computing an orthonormal basis $\boldsymbol{Q}$ for the range of $(\boldsymbol{D}\boldsymbol{D}^T)^q\boldsymbol{D}\boldsymbol{\Omega}$, where $q > 0$, and returns the approximation $\boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{D} \approx \boldsymbol{D}$; we obtain the randomized SVD as a special case by setting $q = 0$. Some remarks:

i) The product $(\boldsymbol{D}\boldsymbol{D}^T)^q\boldsymbol{D}\boldsymbol{\Omega}$ is never formed explicitly. To ensure stability in finite precision arithmetic, $\boldsymbol{Q}$ is obtained by alternating the products with $\boldsymbol{D}$ and its transpose with orthogonalization at each iteration (see Algorithm 1). In exact arithmetic, this does not alter the low-rank approximation.

ii) In exact arithmetic, the approximation $\boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{D}$ depends only on the range of $\boldsymbol{\Omega}$. Hence, replacing $\boldsymbol{\Omega}$ with $\boldsymbol{\Omega}\boldsymbol{X}$ for a non-singular square matrix $\boldsymbol{X}$ has no effect on the approximation.

iii) To improve the performance of the algorithm, one can allow for $\boldsymbol{\Omega}$ to have slightly more than r columns, say $r+5$ or even $2r$ columns. This improves the approximation quality at the cost of more matrix-vector products with $\boldsymbol{D}$. To obtain an approximation of rank r, we can compute the best rank r approximation of $\boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{D}$, which by the orthonormality of $\boldsymbol{Q}$ is $\boldsymbol{Q}\mathcal{T}_r(\boldsymbol{Q}^T\boldsymbol{D})$, requiring only the SVD of the short-and-fat matrix $\boldsymbol{Q}^T\boldsymbol{D}$.

iv) To simplify the analysis, one often assumes that the random matrix $\boldsymbol{\Omega}$ is Gaussian. However, other random matrix formats are also allowed, such as SHRT and subspace injections [**camano2025faster**]. Generally, these formats have weaker guarantees, but perform nearly as good as Gaussians in practice. An explanation for this phenomenon is given in [**camano2025faster**].

Below is a pseudo-code for a stable implementation of subspace iteration. This can be used in place of $\mathcal{T}_r$ in (6).

Algorithm 1: Subspace iteration

Input: Matrix $\boldsymbol{D} \in \mathbb{R}^{m\times n}$, target rank r, number of subspace iterations $2q+1$.
Output: Rank r approximation $\boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{D} \approx \boldsymbol{D}$.

Sample a random matrix $\boldsymbol{\Omega} \in \mathbb{R}^{n\times r}$.
$\boldsymbol{Q} = \operatorname{orth}(\boldsymbol{D}\boldsymbol{\Omega})$.
for $k = 1 \to q$ **do**
    $\boldsymbol{Q} = \operatorname{orth}(\boldsymbol{D}^T\boldsymbol{Q})$;
    $\boldsymbol{Q} = \operatorname{orth}(\boldsymbol{D}\boldsymbol{Q})$
return $\boldsymbol{Q}(\boldsymbol{Q}^T\boldsymbol{D})$ in factored form.

David: Note for self: Note that if $\boldsymbol{P} = \operatorname{orth}(\boldsymbol{D}^T\boldsymbol{\Omega})$ and $\boldsymbol{Q} = \operatorname{orth}(\boldsymbol{D}\boldsymbol{P})$ we have $\boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{A}\boldsymbol{V}\boldsymbol{V}^T = \boldsymbol{A}\boldsymbol{V}\boldsymbol{V}^T$.

In the subsequent discussion, we will assume that $q = 0$ so that Algorithm 1 simplifies to the randomized SVD.

Using the randomized SVD in place of (6) yields the following scheme.

Algorithm 2: (6) with the randomized SVD

Input: Entry access to $\nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W})$, target rank r, number of iterations T, learning rate λ, a low-rank initialization $\boldsymbol{\Delta W}^0 \in \mathbb{R}^{m\times n}$.
Output: Low-rank matrices $\boldsymbol{\Delta W}^t = \boldsymbol{Q}_t\boldsymbol{R}_t$.

Sample a random $\boldsymbol{\Omega} \in \mathbb{R}^{n\times r}$.
Initialize $\boldsymbol{Y}_0 = \boldsymbol{\Delta W}^0\boldsymbol{\Omega}$
for $t = 0 \to T-1$ **do**
    Let $\boldsymbol{G}^t = \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W}^t)$ ▷ We only need to compute matvecs with $\boldsymbol{G}^t$.;
    $\boldsymbol{Y}_{t+1} = \boldsymbol{Y}_t - \lambda\boldsymbol{G}^t\boldsymbol{\Omega}$ ▷ If $\boldsymbol{D}^{t+1} := \boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t$ then $\boldsymbol{Y}_{t+1} = \boldsymbol{D}^{t+1}\boldsymbol{\Omega}$;
    $\boldsymbol{Q}_{t+1} = \operatorname{orth}(\boldsymbol{Y}_{t+1})$;
    $\boldsymbol{R}_{t+1} = \boldsymbol{Q}_{t+1}^T\left(\boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t\right) = \boldsymbol{Q}_{t+1}^T\boldsymbol{Q}_t\boldsymbol{R}_t - \lambda\boldsymbol{Q}^T\boldsymbol{G}^t$ ▷ $\boldsymbol{R}_{t+1} = \boldsymbol{Q}_{t+1}^T\boldsymbol{D}^{t+1}$;
    Let $\boldsymbol{\Delta W}^{t+1} = \boldsymbol{Q}_{t+1}\boldsymbol{R}_{t+1}$.

David: We can replace $\boldsymbol{Y}_{t+1} = \boldsymbol{Y}_t - \lambda\boldsymbol{G}^t\boldsymbol{\Omega}$ with $\boldsymbol{Y}_{t+1} = \boldsymbol{Q}_t\boldsymbol{R}_t\boldsymbol{\Omega} - \lambda\boldsymbol{G}^t\boldsymbol{\Omega}$. The former is a bit cheaper, since we do not need to compute $\boldsymbol{Q}_t(\boldsymbol{R}_t\boldsymbol{\Omega})$, but it requires storing $\boldsymbol{Y}_t$. The latter requires a bit more operations, but once $\boldsymbol{Q}_{t+1}$ is computed we can discard $\boldsymbol{Y}_{t+1}$. These are all tall-and-skinny matrices, so maybe it does not matter at all. In this case, we can ignore the following discussion. From this formulation of the algorithm it is not immediate that it is equivalent to the update

scheme (6), except $\mathcal{T}_r$ is replaced with the randomized SVD. To see this, note that by induction we have

$$\boldsymbol{Y}_{t+1} = \boldsymbol{\Delta W}^0 \boldsymbol{\Omega} - \lambda \sum_{i=0}^{t} \boldsymbol{G}^i \boldsymbol{\Omega}.$$

However, because of the rank r truncations

$$\boldsymbol{D}^{t+1} \neq \boldsymbol{\Delta W}^0 - \lambda \sum_{i=0}^{t} \boldsymbol{G}^i.$$

It is therefore not clear why Algorithm 2 applies the randomized SVD to $\boldsymbol{D}^{t+1}$ at every iteration. However, the following result guarantees that $\boldsymbol{D}^{t+1}\boldsymbol{\Omega} = \boldsymbol{Y}_{t+1}$ and, consequently, Algorithm 2 does indeed apply the randomized SVD to $\boldsymbol{D}^{t+1}$.

Lemma 3.1. Using the notation of Algorithm 2, then for each t we have

$$\boldsymbol{D}^{t+1}\boldsymbol{\Omega} = \boldsymbol{Y}_{t+1}.$$

Hence, Algorithm 2 applies the randomized SVD to $\boldsymbol{D}^{t+1}$ at each iteration.

Proof. The proof is by induction. When $t = 0$, the result is immediate. For the inductive step, we have

$$\begin{aligned}
\boldsymbol{Y}_{t+1} &= \boldsymbol{Y}_t - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} \\
&= \boldsymbol{D}^t \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && \text{(Inductive hypothesis)} \\
&= \boldsymbol{Q}_t \boldsymbol{Q}_t^T \boldsymbol{D}^t \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\boldsymbol{Q}_t \text{ is an o.n.b. for } \boldsymbol{D}^t\boldsymbol{\Omega}) \\
&= \boldsymbol{\Delta W}^t \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\boldsymbol{Q}_t \boldsymbol{Q}_t^T \boldsymbol{D}^t = \boldsymbol{\Delta W}^t) \\
&= \boldsymbol{D}^{t+1} \boldsymbol{\Omega}, && (\boldsymbol{\Delta W}^t - \lambda \boldsymbol{G}^t = \boldsymbol{D}^{t+1})
\end{aligned}$$

as required. □

3.2 Generalized Nyström method

A drawback with subspace iteration and the randomized SVD is that subsequent computations depend on earlier ones: you need to compute $\boldsymbol{Q}$ before you compute $\boldsymbol{Q}^T\boldsymbol{D}$. This makes the algorithm less suitable in parallel computing environments. Instead, one can resort to the *generalized Nyström approximation* [**nakatsukasa2020fast**]. It first samples two random matrices $\boldsymbol{\Omega} \in \mathbb{R}^{n\times r}$ and $\boldsymbol{\Psi} \in \mathbb{R}^{m\times r+s}$, where s is a small oversampling parameter; a sensible default value is $s = 5$.[1] It then proceeds by returning the approximation

$$\boldsymbol{D}\boldsymbol{\Omega} \left(\boldsymbol{\Psi}^T \boldsymbol{D}\boldsymbol{\Omega}\right)^{\dagger} \boldsymbol{\Psi}^T \boldsymbol{D} \approx \boldsymbol{D}, \tag{7}$$

[1] As highlighted in Section 3.1, to improve the accuracy of the approximation one can allow $\boldsymbol{\Omega}$ to have slightly more than r columns, say $r+p$. In this case, $\boldsymbol{\Psi}$ should have $r+p+s$ columns. The rule of thumb is $\boldsymbol{\Psi}$ should always have a few more columns than $\boldsymbol{\Omega}$.

where † denotes the Moore-Penrose pseudoinverse.[2] The generalized Nyström approximation is (provably) less accurate than subspace iteration for *any* q [**funnystrom2**], but is easier to parallelize: the triple $(\boldsymbol{D\Omega}, \boldsymbol{\Psi}^T\boldsymbol{D\Omega}, \boldsymbol{\Psi}^T\boldsymbol{D})$ can be computed in parallel. Hence, we can use (7) in place of $\mathcal{T}_r$ in (6). Applying the generalized Nyström approximation in place of $\mathcal{T}_r$ yields Algorithm 3.

Algorithm 3: (6) with generalized Nyström

Input: Entry access to $\nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W})$, target rank r, oversampling parameter s, number of iterations T, learning rate λ, a low-rank initialization $\boldsymbol{\Delta W}^0 \in \mathbb{R}^{m\times n}$.

Output: Low-rank matrices $\boldsymbol{\Delta W}^t = \boldsymbol{Y}_t \boldsymbol{M}_t^\dagger \boldsymbol{Z}_t^T$.

1: Sample two random matrices $\boldsymbol{\Omega} \in \mathbb{R}^{n\times r}$ and $\boldsymbol{\Psi} \in \mathbb{R}^{m\times r+s}$.

2: Initialize $\boldsymbol{Y}_0 = \boldsymbol{\Delta W}^0\boldsymbol{\Omega}, \boldsymbol{M}_0 = \boldsymbol{\Psi}^T\boldsymbol{\Delta W}^0\boldsymbol{\Omega}, \boldsymbol{Z}_0^T = \boldsymbol{\Psi}^T\boldsymbol{\Delta W}^0$.

3: **for** $t = 0 \to T-1$ **do**

$\boldsymbol{G}_t = \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W}^t)$ ▷ We only need matvecs with $\boldsymbol{G}^t$;

$\boldsymbol{Y}_{t+1} = \boldsymbol{Y}_t - \lambda \boldsymbol{G}^t\boldsymbol{\Omega}$ ▷ If $\boldsymbol{D}^{t+1} = \boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t$ then $\boldsymbol{Y}_{t+1} = \boldsymbol{D}^{t+1}\boldsymbol{\Omega}$;

$\boldsymbol{Z}_{t+1}^T = \boldsymbol{M}_t\boldsymbol{M}_t^\dagger\boldsymbol{Z}_t^T - \lambda\boldsymbol{\Psi}^T\boldsymbol{G}^t$ ▷ $\boldsymbol{Z}_{t+1}^T = \boldsymbol{\Psi}^T\boldsymbol{D}^{t+1}$;

$\boldsymbol{M}_{t+1} = \boldsymbol{\Psi}^T\boldsymbol{Y}_{t+1} = \boldsymbol{M}_t - \lambda\boldsymbol{\Psi}^T\boldsymbol{G}^t\boldsymbol{\Omega}$ ▷ $\boldsymbol{M}_{t+1} = \boldsymbol{\Psi}^T\boldsymbol{D}^{t+1}\boldsymbol{\Omega}$;

Let $\boldsymbol{\Delta W}^{t+1} = \boldsymbol{Y}_{t+1}\boldsymbol{M}_{t+1}^\dagger\boldsymbol{Z}_{t+1}^T$.

Note that the triple $(\boldsymbol{G}^t\boldsymbol{\Omega}, \boldsymbol{\Psi}^T\boldsymbol{G}^t\boldsymbol{\Omega}, \boldsymbol{\Psi}^T\boldsymbol{G}^t)$ can be computed by only looking at each entry of $\boldsymbol{G}^t$ once, and then discard it.[3]

As with Algorithm 2, it is not immediate why $\boldsymbol{Y}_{t+1}\boldsymbol{M}_{t+1}^\dagger\boldsymbol{Z}_{t+1}^T$ is the generalized Nyström approximation of $\boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t$. Once again, this can be proven by induction.

Lemma 3.2. Using the notation of Algorithm 3, define $\boldsymbol{D}^{t+1} = \boldsymbol{\Delta W}^t - \lambda\boldsymbol{G}^t$. Assume that $\boldsymbol{M}_t$ is full-rank for each t. Then, for each t we have

$$\boldsymbol{D}^{t+1}\boldsymbol{\Omega}(\boldsymbol{\Psi}^T\boldsymbol{D}^{t+1}\boldsymbol{\Omega})^\dagger\boldsymbol{\Psi}^T\boldsymbol{D}^{t+1} = \boldsymbol{Y}_{t+1}\boldsymbol{M}_{t+1}^\dagger\boldsymbol{Z}_{t+1}^T.$$

Hence, Algorithm 3 applies the generalized Nyström approximation to $\boldsymbol{D}^{t+1}$ at each iteration.

Proof. It is sufficient to prove

$$(\boldsymbol{Y}_t, \boldsymbol{M}_t, \boldsymbol{Z}_t^T) = (\boldsymbol{D}^t\boldsymbol{\Omega}, \boldsymbol{\Psi}^T\boldsymbol{D}^t\boldsymbol{\Omega}, \boldsymbol{\Psi}^t\boldsymbol{D}^t) \tag{8}$$

for each t. This will be proven by induction. For $t = 0$ the result is immediate. For the inductive

[2] If $\boldsymbol{Q}$ is an orthonormal basis for the range of $\boldsymbol{D\Omega}$, one can verify that the generalized Nyström approximation in (7) is equivalent to $\boldsymbol{Q}(\boldsymbol{\Psi}^T\boldsymbol{Q})^\dagger\boldsymbol{\Psi}^T\boldsymbol{D}$.

[3] We can write $\boldsymbol{G}_t = \sum_{i,j}(\boldsymbol{G}_t)_{ij}\boldsymbol{E}_{ij}$, where $\boldsymbol{E}_{ij}$ is the matrix that is zero everywhere except at the (i,j) entry, where it is 1.

step, we have

$$\begin{aligned}
\boldsymbol{Y}_{t+1} &= \boldsymbol{Y}_t - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} \\
&= \boldsymbol{Y}_t \boldsymbol{M}_t^\dagger \boldsymbol{M}_t - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\boldsymbol{M}_t \text{ has full rank}) \\
&= \boldsymbol{Y}_t \boldsymbol{M}_t^\dagger \boldsymbol{\Psi}^T \boldsymbol{D}^t \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\text{Inductive hypothesis}) \\
&= \boldsymbol{Y}_t \boldsymbol{M}_t^\dagger \boldsymbol{Z}_t^T \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\text{Inductive hypothesis}) \\
&= \boldsymbol{\Delta W}^t \boldsymbol{\Omega} - \lambda \boldsymbol{G}^t \boldsymbol{\Omega} && (\text{Definition of } \boldsymbol{\Delta W}^t) \\
&= \boldsymbol{D}^{t+1} \boldsymbol{\Omega} && (\text{Definition of } \boldsymbol{D}^{t+1})
\end{aligned}$$

Hence, $\boldsymbol{Y}_{t+1} = \boldsymbol{D}^{t+1}\boldsymbol{\Omega}$. We immediately get $\boldsymbol{M}_{t+1} = \boldsymbol{\Psi}^T \boldsymbol{Y}_{t+1} = \boldsymbol{\Psi}^T \boldsymbol{D}^{t+1}\boldsymbol{\Omega}$. We will now proceed with proving $\boldsymbol{Z}_{t+1} = \boldsymbol{\Psi}^T \boldsymbol{D}^{t+1}$. We have

$$\begin{aligned}
\boldsymbol{Z}_{t+1}^T &= \boldsymbol{M}_t \boldsymbol{M}_t^\dagger \boldsymbol{Z}_t^T - \lambda \boldsymbol{\Psi}^T \boldsymbol{G}^t \\
&= \boldsymbol{\Psi}^T \boldsymbol{Y}_t \boldsymbol{M}_t^\dagger \boldsymbol{Z}_t^T - \lambda \boldsymbol{\Psi}^T \boldsymbol{G}^t && (\text{Inductive hypothesis}) \\
&= \boldsymbol{\Psi}^T \boldsymbol{\Delta W}^t - \lambda \boldsymbol{\Psi}^T \boldsymbol{G}^t && (\text{Definition of } \boldsymbol{\Delta W}^t) \\
&= \boldsymbol{\Psi}^T \boldsymbol{D}^{t+1}, && (\text{Definition of } \boldsymbol{D}^{t+1})
\end{aligned}$$

as required. Hence, by induction, (8) holds for all t □

3.3 Reusing sketches

In this section, we discuss how randomized low-rank approximation techniques can be used to compute low-rank approximations to $\boldsymbol{\Delta W}^t - \lambda \boldsymbol{G}^t$, where $\boldsymbol{\Delta W}^t = \boldsymbol{U}_t \boldsymbol{S}_t \boldsymbol{V}_t^T$ and we only have access to $\boldsymbol{G}^t \boldsymbol{V}_t$ and $\boldsymbol{U}_t^T \boldsymbol{G}^t$.

Remark 3.3. **David:** In our discussions, we noted that given a low-rank factorization $\boldsymbol{\Delta W}^t = \boldsymbol{L}_t \boldsymbol{R}_t$ we immediately have access to $\boldsymbol{G}^t \boldsymbol{R}_t^T$ and $\boldsymbol{L}_t^T \boldsymbol{G}^t$. By orthogonalizing $\boldsymbol{U}_t = \text{orth}(\boldsymbol{L}_t)$ and $\boldsymbol{V}_t = \text{orth}(\boldsymbol{R}_t^T)$ we have $\boldsymbol{G}^t \boldsymbol{V}_t = \boldsymbol{G}^t \boldsymbol{R}_t^T (\boldsymbol{V}_t^T \boldsymbol{R}_t^T)^{-1}$ and $\boldsymbol{U}_t^T \boldsymbol{G}^t = (\boldsymbol{L}_t^T \boldsymbol{U}_t^T)^{-1} \boldsymbol{L}_t^T \boldsymbol{G}^t$. We would need to perform some form of orthogonalization, otherwise repeatedly applying multiplications $\boldsymbol{G}^t \boldsymbol{R}_t^T$ and $\boldsymbol{L}_t^T \boldsymbol{G}^t$ would lead to a loss of precision in floating-point arithmetic as t increases. We therefore assume that $\boldsymbol{G}^t$ is multiplied by matrices with orthonormal columns.

Consider the update $\boldsymbol{D}^{t+1} = \boldsymbol{\Delta W}^t - \lambda \boldsymbol{G}^t$, where $\boldsymbol{\Delta W}^t = \boldsymbol{L}_t \boldsymbol{R}_t = \boldsymbol{U}_t \boldsymbol{S}_t \boldsymbol{V}_t^T$. Here, $\boldsymbol{U}_t$ and $\boldsymbol{V}_t$ have r orthonormal columns and $\boldsymbol{S}_t \in \mathbb{R}^{r \times r}$ is full-rank (not necessarily diagonal). The factors $\boldsymbol{L}_t$ and $\boldsymbol{R}_t$ can be written as

$$\boldsymbol{L}_t = \boldsymbol{U}_t \boldsymbol{X}_t, \quad \boldsymbol{R}_t = \boldsymbol{K}_t^T \boldsymbol{V}_t^T,$$

for some full rank $\boldsymbol{X}_t$ and $\boldsymbol{K}_t$. We can compute the generalized Nyström approximation of $\boldsymbol{D}^{t+1}$ using $\boldsymbol{L}_t$ and $\boldsymbol{R}_t$

$$\begin{aligned}
\boldsymbol{\Delta W}^{t+1} &= \boldsymbol{D}^{t+1} \boldsymbol{R}_t^T (\boldsymbol{L}_t^T \boldsymbol{D}^{t+1} \boldsymbol{R}_t^T)^\dagger \boldsymbol{L}_t^T \boldsymbol{D}^{t+1} \\
&= \boldsymbol{D}^{t+1} \boldsymbol{V}_t \boldsymbol{K}_t (\boldsymbol{X}_t^T \boldsymbol{U}_t^T \boldsymbol{D}^{t+1} \boldsymbol{V}_t \boldsymbol{K}_t)^\dagger \boldsymbol{X}_t^T \boldsymbol{U}_t^T \boldsymbol{D}^{t+1}.
\end{aligned}$$

Assume that $\operatorname{rank}(\boldsymbol{U}_t^T \boldsymbol{D}^{t+1} \boldsymbol{V}_t) = r$; otherwise the rank will decrease. Then,

$$\boldsymbol{D}^{t+1}\boldsymbol{V}_t\boldsymbol{K}_t(\boldsymbol{X}_t^T\boldsymbol{U}_t^T\boldsymbol{D}^{t+1}\boldsymbol{V}_t\boldsymbol{K}_t)^\dagger\boldsymbol{X}_t^T\boldsymbol{U}_t^T\boldsymbol{D}^{t+1} = \boldsymbol{D}^{t+1}\boldsymbol{V}_t(\boldsymbol{U}_t^T\boldsymbol{D}^{t+1}\boldsymbol{V}_t)^\dagger\boldsymbol{U}_t^T\boldsymbol{D}^{t+1}.$$

Hence, provided $\operatorname{rank}(\boldsymbol{U}_t^T \boldsymbol{D}^{t+1} \boldsymbol{V}_t) = r$, the generalized Nyström approximation using $\boldsymbol{R}_t^T$ and $\boldsymbol{L}_t$ as sketches is equivalent to using orthonormal bases for their ranges. We can compute

$$\boldsymbol{Y}_{t+1} = \boldsymbol{D}^{t+1}\boldsymbol{V}_t, \quad \boldsymbol{Z}_{t+1}^T = \boldsymbol{U}_t^T\boldsymbol{D}^{t+1}, \quad \boldsymbol{M}_{t+1} = \boldsymbol{U}_t^T\boldsymbol{D}^{t+1}\boldsymbol{V}_t,$$

Next, compute orthonormal bases $\boldsymbol{U}_{t+1}$ for $\boldsymbol{Y}_{t+1}$ and $\boldsymbol{V}_{t+1}$ for $\boldsymbol{Z}_{t+1}$ (using e.g. Polar Express). Then,

$$\begin{aligned}
\boldsymbol{\Delta W}^{t+1} &= \boldsymbol{Y}_{t+1}\boldsymbol{M}_{t+1}^\dagger\boldsymbol{Z}_{t+1}^T \\
&= \boldsymbol{U}_{t+1}(\boldsymbol{U}_{t+1}^T\boldsymbol{Y}_{t+1})\boldsymbol{M}_{t+1}^\dagger(\boldsymbol{Z}_{t+1}^T\boldsymbol{V}_{t+1})\boldsymbol{V}_{t+1}^T \\
&= \boldsymbol{U}_{t+1}\boldsymbol{S}_{t+1}\boldsymbol{V}_{t+1}^T. \qquad (\boldsymbol{S}_{t+1} = (\boldsymbol{U}_{t+1}^T\boldsymbol{Y}_{t+1})\boldsymbol{M}_{t+1}^\dagger(\boldsymbol{Z}_{t+1}^T\boldsymbol{V}_{t+1}))
\end{aligned}$$

This yields the following algorithm.

Algorithm 4: (6) with generalized Nyström and reused sketches

Input: Target rank r, number of iterations T, learning rate λ, a low-rank initialization $\boldsymbol{\Delta W}^0 = \boldsymbol{U}_0\boldsymbol{S}_0\boldsymbol{V}_0^T$.

Output: Approximations $\boldsymbol{U}_t\boldsymbol{S}_t\boldsymbol{V}_t^T$ to $\boldsymbol{\Delta W}^t$ in (6).

1: Initialize a low-rank factorization $\boldsymbol{\Delta W}^0 = \boldsymbol{U}_0\boldsymbol{S}_0\boldsymbol{V}_0^T$

2: **for** $t = 0 \to T-1$ **do**

$\boldsymbol{G}_t = \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{\Delta W}^t)$ ▷ Do not form explicitly;

$\boldsymbol{Y}_{t+1} = \boldsymbol{\Delta W}^t\boldsymbol{V}_t - \lambda\boldsymbol{G}^t\boldsymbol{V}_t = \boldsymbol{U}_t\boldsymbol{S}_t - \lambda\boldsymbol{G}^t\boldsymbol{V}_t$;

$\boldsymbol{M}_{t+1} = \boldsymbol{U}_t^T\boldsymbol{\Delta W}^t\boldsymbol{V}_t - \lambda\boldsymbol{U}_t^T\boldsymbol{G}^t\boldsymbol{V}_t = \boldsymbol{S}_t - \lambda\boldsymbol{U}_t^T\boldsymbol{G}^t\boldsymbol{V}_t$;

$\boldsymbol{Z}_{t+1}^T = \boldsymbol{U}_t^T\boldsymbol{\Delta W}^t - \lambda\boldsymbol{U}_t^T\boldsymbol{G}^t = \boldsymbol{S}_t\boldsymbol{V}_t^T - \lambda\boldsymbol{U}_t^T\boldsymbol{G}^t$;

$\boldsymbol{U}_{t+1} = \operatorname{orth}(\boldsymbol{Y}_{t+1})$;

$\boldsymbol{V}_{t+1} = \operatorname{orth}(\boldsymbol{Z}_{t+1})$;

$\boldsymbol{S}_{t+1} = \boldsymbol{U}_{t+1}^T\boldsymbol{Y}_{t+1}\boldsymbol{M}_{t+1}^\dagger\boldsymbol{Z}_{t+1}^T\boldsymbol{V}_{t+1}$.

David: Some junk notes. Tried to prove some error bound

Theorem 3.4. Let $\boldsymbol{B} = \boldsymbol{A} + \boldsymbol{E} \in \mathbb{R}^{m\times n}$, where $\operatorname{rank}(\boldsymbol{A}) \le r$ and $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$ is the rank-reduced SVD of $\boldsymbol{A}$. Consider the randomized SVD applied to $\boldsymbol{B}$ with $\boldsymbol{V} \in \mathbb{R}^{n\times r}$ as a sketch matrix:**David:** this is not a great term in this setting, since there will be nothing random here, but I am not sure what else to call it. One step of subspace iteration?

$$\boldsymbol{Q} = \operatorname{orth}(\boldsymbol{B}\boldsymbol{V}), \quad \boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{B} \approx \boldsymbol{B}.$$

Then,

$$\|\boldsymbol{B} - \boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{B}\| \le 2\|\boldsymbol{E}\|,$$

for any norm $\|\cdot\|$ satisfying $\|\boldsymbol{C}\boldsymbol{D}\| \le \max\{\|\boldsymbol{C}\|_2\|\boldsymbol{D}\|, \|\boldsymbol{C}\|\|\boldsymbol{D}\|_2\}$.

Proof. By the triangle inequality, the characterization of the norm $\|\cdot\|$, and $\|\boldsymbol{I} - \boldsymbol{Q}\boldsymbol{Q}^T\|_2 = 1$ we have

$$\|\boldsymbol{B} - \boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{B}\| \leq \|\boldsymbol{A} - \boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{A}\| + \|\boldsymbol{E}\|.$$

Using that $\operatorname{rank}(\boldsymbol{A}) \leq r$ and applying the argument of [**hodlr**] gives $\|\boldsymbol{A} - \boldsymbol{Q}\boldsymbol{Q}^T\boldsymbol{A}\| \leq \|\boldsymbol{E}\boldsymbol{V}\| \leq \|\boldsymbol{E}\|$, where we once again used the characterization of $\|\cdot\|$. Combining the two inequalities yields the desired result. □

4 Linearizing and Least-Squares approach

We will approximately solve (4) by linearizing the quadratic term $\boldsymbol{L}\boldsymbol{R}$, that is

$$\boldsymbol{L}\boldsymbol{R} - \boldsymbol{L}_t\boldsymbol{R}_t \approx \boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t. \tag{9}$$

Using this approximation in (4) gives

$$\begin{aligned}\boldsymbol{\Delta W}^{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r},\boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} \; & \langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\rangle \\ & + \frac{1}{2\lambda}\|\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\|_F^2. \end{aligned} \tag{10}$$

This problem always admits a least-norm solution because it is lower bounded. Indeed by completing the squares it can be re-written as follows, where we use $\boldsymbol{G} = \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t\boldsymbol{R}_t)$ as a short-hand.

Lemma 4.1 (Least-norm matrix solution via projector decomposition and a core Sylvester solve)**.** Given $\boldsymbol{L}_t \in \mathbb{R}^{m\times r}$, $\boldsymbol{R}_t \in \mathbb{R}^{r\times n}$, and $\boldsymbol{G} \in \mathbb{R}^{m\times n}$, consider

$$\min_{\boldsymbol{L},\boldsymbol{R}} \left\| \boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t + \lambda\boldsymbol{G} \right\|_F^2. \tag{11}$$

Assume $\boldsymbol{\Sigma}_{\boldsymbol{L}} := \boldsymbol{L}_t^\top\boldsymbol{L}_t \succ 0$ and $\boldsymbol{\Sigma}_{\boldsymbol{R}} := \boldsymbol{R}_t\boldsymbol{R}_t^\top \succ 0$. Then the least-norm minimizer $(\boldsymbol{L}_{t+1}, \boldsymbol{R}_{t+1})$ is given by

$$\boldsymbol{L}_{t+1} = \boldsymbol{L}_t - (\lambda\boldsymbol{G}_t\boldsymbol{R}_t^\top + \boldsymbol{L}_t\boldsymbol{K}^\star)\boldsymbol{\Sigma}_R^{-1} \tag{12}$$

$$\boldsymbol{R}_{t+1} = \boldsymbol{R}_t - \boldsymbol{\Sigma}_L^{-1}(\lambda\boldsymbol{L}_t^\top\boldsymbol{G}_t + \boldsymbol{K}^\star\boldsymbol{R}_t) \tag{13}$$

where $\boldsymbol{K}^\star$ is the solution to the Sylvester equation

$$\boldsymbol{K}\boldsymbol{\Sigma}_R + \boldsymbol{\Sigma}_L\boldsymbol{K} = -\lambda\boldsymbol{L}_t^\top\boldsymbol{G}_t\boldsymbol{R}_t^\top \tag{14}$$

See Remark 4.2 on how to solve the Sylvester equation in $\mathcal{O}(r^3)$ operations.

Proof. David: By vectorizing and taking normal equations, a similar problem has been pointed out in *The ubiquitous Kronecker product* equation (6) Define the linear of the map $(\boldsymbol{L}, \boldsymbol{R}) \mapsto \boldsymbol{L}\boldsymbol{R}$ at $(\boldsymbol{L}_t, \boldsymbol{R}_t)$ to be $\mathcal{J}_t[\Delta\boldsymbol{L}, \Delta\boldsymbol{R}] = \Delta\boldsymbol{L}\boldsymbol{R}_t + \boldsymbol{L}_t\Delta\boldsymbol{R}$. With this notation (11) is given by

$$\underset{\boldsymbol{Z}}{\operatorname{argmin}} \|\boldsymbol{Z} + \lambda\boldsymbol{G}_t\|_F^2 \;\text{ s.t. }\; \boldsymbol{Z} = J_t[\Delta\boldsymbol{L}, \Delta\boldsymbol{R}].$$

All solutions satisfy the normal equations

$$\mathcal{J}_t^\star \mathcal{J}_t[\Delta \boldsymbol{L}, \Delta \boldsymbol{R}] = -\lambda J_t^\star[\boldsymbol{G}_t].$$

The adjoint $\mathcal{J}_t^\star(\boldsymbol{M}) = (\boldsymbol{M}\boldsymbol{R}_t^\top, \boldsymbol{L}_t^\top \boldsymbol{M})$. Plugging this in to the normal equations gives us

$$\begin{aligned}\Delta \boldsymbol{L}\boldsymbol{R}_t\boldsymbol{R}_t^\top + \boldsymbol{L}_t\Delta \boldsymbol{R}\boldsymbol{R}_t^\top &= -\lambda \boldsymbol{G}_t\boldsymbol{R}_t^\top \\ \boldsymbol{L}_t^\top\Delta \boldsymbol{L}\boldsymbol{R}_t + \boldsymbol{L}_t^\top\boldsymbol{L}_t\Delta \boldsymbol{R} &= -\lambda \boldsymbol{L}_t^\top\boldsymbol{G}_t\end{aligned}$$

which we abbreviate as

$$\begin{aligned}\Delta \boldsymbol{L}\boldsymbol{\Sigma}_R + \boldsymbol{L}_t\boldsymbol{X} &= -\lambda \boldsymbol{G}_t\boldsymbol{R}_t^\top \\ \boldsymbol{Y}\boldsymbol{R}_t + \boldsymbol{\Sigma}_L\Delta \boldsymbol{R} &= -\lambda \boldsymbol{L}_t^\top\boldsymbol{G}_t,\end{aligned} \tag{15}$$

where we define

$$\boldsymbol{\Sigma}_R = \boldsymbol{R}_t\boldsymbol{R}_t^\top,\ \boldsymbol{\Sigma}_L = \boldsymbol{L}_t^\top\boldsymbol{L}_t,\ \boldsymbol{X} = \Delta \boldsymbol{R}\boldsymbol{R}_t^\top,\ \boldsymbol{Y} = \boldsymbol{L}_t^\top\Delta \boldsymbol{L}.$$

We will first solve for $\boldsymbol{X}$ and $\boldsymbol{Y}$, and then using $\boldsymbol{X}$ and $\boldsymbol{Y}$, we will then solve for $\Delta \boldsymbol{L}$ and $\Delta \boldsymbol{R}$ in (15).

To solve for $\boldsymbol{X}$ and $\boldsymbol{Y}$, multiplying the top equation by $\boldsymbol{L}_t^\top$ on the left (or the bottom by $\boldsymbol{R}_t^\top$ on the right), gives an equation in only $\boldsymbol{X}$ and $\boldsymbol{Y}$:

$$\boldsymbol{Y}\boldsymbol{\Sigma}_R + \boldsymbol{\Sigma}_L\boldsymbol{X} = -\lambda \boldsymbol{L}_t^\top\boldsymbol{G}_t\boldsymbol{R}_t^\top.$$

Note that $\mathcal{J}_t$ has a nullspace spanned by $(\boldsymbol{L}_t S, -S\boldsymbol{R}_t)$ (assuming $\boldsymbol{L}_t$ and $\boldsymbol{R}_t$ are full-rank). Therefore to choose a minimum norm solution (standard inner product) we require

$$\langle(\Delta \boldsymbol{L}, \Delta \boldsymbol{R}), (\boldsymbol{L}_t S, -S\boldsymbol{R}_t)\rangle \ = \ \langle \Delta \boldsymbol{L}, \boldsymbol{L}_t S\rangle + \langle \Delta \boldsymbol{R}, -S\boldsymbol{R}_t\rangle \ = \ \left\langle \boldsymbol{L}_t^\top\Delta \boldsymbol{L} - \Delta \boldsymbol{R}\boldsymbol{R}_t^\top, S\right\rangle \ = \ 0.$$

Since this holds for every S, we have that

$$\boldsymbol{L}_t^\top\Delta \boldsymbol{L} - \Delta \boldsymbol{R}\boldsymbol{R}_t^\top \quad \Leftrightarrow \quad \boldsymbol{X} = \boldsymbol{Y}$$

Therefore we need to solve the Sylvester equation

$$\boldsymbol{K}\boldsymbol{\Sigma}_R + \boldsymbol{\Sigma}_L\boldsymbol{K} = -\lambda \boldsymbol{L}_t^\top\boldsymbol{G}_t\boldsymbol{R}_t^\top$$

for $\boldsymbol{K}$. This an $r \times r$ matrix equation so $O(r^3)$ to solve. If the solution is $\boldsymbol{K}^\star$ then

$$\begin{aligned}\Delta \boldsymbol{L}^\star &= -(\lambda \boldsymbol{G}_t\boldsymbol{R}_t^\top + \boldsymbol{L}_t\boldsymbol{K}^\star)\boldsymbol{\Sigma}_R^{-1} \\ \Delta \boldsymbol{R}^\star &= -\boldsymbol{\Sigma}_L^{-1}(\lambda \boldsymbol{L}_t^\top\boldsymbol{G}_t + \boldsymbol{K}^\star\boldsymbol{R}_t)\end{aligned}$$

This requires computing $\boldsymbol{\Sigma}_R^{-1}$ and $\boldsymbol{\Sigma}_L^{-1}$ which are $r \times r$ so this is also $O(r^3)$. □

Remark 4.2 (Solving the Sylvester Equation)**.** We can solve the Sylvester equation (14), that is

$$\boldsymbol{K}\boldsymbol{\Sigma}_R + \boldsymbol{\Sigma}_L\boldsymbol{K} = -\boldsymbol{\Sigma} \tag{16}$$

in $\mathcal{O}(r^3)$ by computing the SVD of $\boldsymbol{\Sigma_L}$ and $\boldsymbol{\Sigma_R}$ denoted by

$$\boldsymbol{\Sigma_L} = \boldsymbol{Q_L}\,\mathrm{diag}(\alpha)\boldsymbol{Q_L}^\top, \qquad \text{and} \qquad \boldsymbol{\Sigma_R} = \boldsymbol{Q_R}\,\mathrm{diag}(\beta)\boldsymbol{Q_R}^\top.$$

Left and right multiplying the Sylvester equation (16) by $\boldsymbol{Q}_{\boldsymbol{L}}^\top$ and $\boldsymbol{Q}_{\boldsymbol{R}}$, respectively gives

$$\boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{K} \boldsymbol{Q}_{\boldsymbol{R}} \operatorname{diag}(\beta) + \operatorname{diag}(\alpha) \boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{K} \boldsymbol{Q}_{\boldsymbol{R}} = -\boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{\Sigma} \boldsymbol{Q}_{\boldsymbol{R}} \tag{17}$$

Now let

$$\boldsymbol{X} := \boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{K} \boldsymbol{Q}_{\boldsymbol{R}}, \qquad \text{and} \qquad \widetilde{\boldsymbol{\Sigma}} := \boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{\Sigma}\, \boldsymbol{Q}_{\boldsymbol{R}}$$

and note that (17) can be re-written as

$$(\alpha_i + \beta_j) \boldsymbol{X}_{ij} = -\widetilde{\boldsymbol{\Sigma}}_{ij} \quad \Leftrightarrow \quad \boldsymbol{X}_{ij} = -\frac{\widetilde{\boldsymbol{\Sigma}}_{ij}}{\alpha_i + \beta_j}.$$

In matrix notation, and returning to $\boldsymbol{K} = \boldsymbol{Q}_L \boldsymbol{X} \boldsymbol{Q}_R$ we have the solution

$$\boldsymbol{K}^\star \;=\; -\boldsymbol{Q}_L \left((\boldsymbol{Q}_{\boldsymbol{L}}^\top \boldsymbol{\Sigma} \boldsymbol{Q}_{\boldsymbol{R}}) \oslash (\alpha \mathbf{1}^\top + \mathbf{1} \beta^\top) \right) \boldsymbol{Q}_R \tag{18}$$

Robert: Everything below here is possibly obsolete, and not worth reading.

4.1 Alternating and splitting

We will approximately solve (4) by alternating between solving in $\boldsymbol{L}$ and $\boldsymbol{R}$ as follows

$$\boldsymbol{L}_{t+1} = \operatorname*{argmin}_{\boldsymbol{L} \in \mathbb{R}^{m \times r}} F(\boldsymbol{L}_t \boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t), (\boldsymbol{L} - \boldsymbol{L}_t) \boldsymbol{R}_t \rangle + \frac{1}{2\lambda} \|(\boldsymbol{L} - \boldsymbol{L}_t) \boldsymbol{R}_t\|^2, \tag{19}$$

$$\boldsymbol{R}_{t+1} = \operatorname*{argmin}_{\boldsymbol{R} \in \mathbb{R}^{r \times n}} F(\boldsymbol{L}_t \boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{L}_t (\boldsymbol{R} - \boldsymbol{R}_t) \rangle + \frac{1}{2\lambda} \|\boldsymbol{L}_t (\boldsymbol{R} - \boldsymbol{R}_t)\|^2. \tag{20}$$

Now note that the linear terms can be re-written as

$$\begin{aligned} \langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t), (\boldsymbol{L} - \boldsymbol{L}_t) \boldsymbol{R}_t \rangle &= \left\langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t) \boldsymbol{R}_t^\top, \boldsymbol{L} - \boldsymbol{L}_t \right\rangle \\ &= \langle \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{L} - \boldsymbol{L}_t \rangle \end{aligned} \tag{21}$$

where we used that

$$\nabla_{\boldsymbol{L}} F(\boldsymbol{L}\boldsymbol{R}) \Big|_{\boldsymbol{L}\boldsymbol{R} = \boldsymbol{L}_t \boldsymbol{R}_t} = \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t) \boldsymbol{R}_t^\top.$$

Analogously we have that

$$\langle \nabla_{\boldsymbol{\Delta W}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{L}_t (\boldsymbol{R} - \boldsymbol{R}_t) \rangle = \langle \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{R} - \boldsymbol{R}_t \rangle \tag{22}$$

Using the two identities (21) and (22) in (19) and (20) gives

$$\boldsymbol{L}_{t+1} = \operatorname*{argmin}_{\boldsymbol{L} \in \mathbb{R}^{m \times r}} F(\boldsymbol{L}_t \boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{L} - \boldsymbol{L}_t) \rangle + \frac{1}{2\lambda} \|(\boldsymbol{L} - \boldsymbol{L}_t) \boldsymbol{R}_t\|^2, \tag{23}$$

$$\boldsymbol{R}_{t+1} = \operatorname*{argmin}_{\boldsymbol{R} \in \mathbb{R}^{r \times n}} F(\boldsymbol{L}_t \boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t \boldsymbol{R}_t), \boldsymbol{R} - \boldsymbol{R}_t \rangle + \frac{1}{2\lambda} \|\boldsymbol{L}_t (\boldsymbol{R} - \boldsymbol{R}_t)\|^2. \tag{24}$$

Compare the above with (2)–(3). The regularizor now takes into account the product $\boldsymbol{LR}$.

If we use a Frobenius norm in (23)–(24) then we have a closed form solution.

Lemma 4.3. The solution to

$$\boldsymbol{L}_{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r}}{\operatorname{argmin}} \; F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{L} - \boldsymbol{L}_t\rangle + \frac{1}{2\lambda}\|(\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\|_F^2, \tag{25}$$

$$\boldsymbol{R}_{t+1} = \underset{\boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} \; F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{R} - \boldsymbol{R}_t\rangle + \frac{1}{2\lambda}\|\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t)\|_F^2. \tag{26}$$

is given by

$$\boldsymbol{L} = \boldsymbol{L}_t - \lambda \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t\boldsymbol{R}_t)(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger \tag{27}$$

$$\boldsymbol{R} = \boldsymbol{R}_t - \lambda(\boldsymbol{L}_t^\top\boldsymbol{L}_t)^\dagger \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \tag{28}$$

where we use $\boldsymbol{L}_t^\dagger$ to denote the pseudo-inverse of a matrix $\boldsymbol{L}_t$.

Robert: Question: When we look at what happens to the product $\boldsymbol{LR}$ what does this method look like?

Proof. Both updates are given by a convex and smooth program, for which the stationarity conditions

$$\lambda \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t\boldsymbol{R}_t) + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\boldsymbol{R}_t^\top = 0,$$
$$\lambda \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t\boldsymbol{R}_t) + (\boldsymbol{L}_t)^\top\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) = 0.$$

Isolating $\boldsymbol{L}$ and $\boldsymbol{R}$ in the top and bottom equations, and computing the least norm solution, gives (31) and (28), respectively. □

Computing (31)–(28) costs $\mathcal{O}(nr^2 + mr^2 + r^3)$. The nr^2 and mr^2 costs comes from multiplying the matrix multiplication in $(\boldsymbol{L}_t^\top\boldsymbol{L}_t)^\dagger\nabla_{\boldsymbol{R}}F(\boldsymbol{L}_t\boldsymbol{R}_t)$ and $\lambda\nabla_{\boldsymbol{L}}F(\boldsymbol{L}_t\boldsymbol{R}_t)(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger$, respectively. The r^3 cost comes from computing the pseudo-inverses.

If the above is too costly, or we wish to use a norm other than the Frobenius norm (e.g. the operator norm), then we can instead use the "η–trick" in the next section.

Lemma 4.4. The solution to

$$\boldsymbol{L}_{t+1} = \underset{\boldsymbol{L}\in\mathbb{R}^{m\times r}}{\operatorname{argmin}} \; F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{L} - \boldsymbol{L}_t)\rangle + \frac{1}{2\lambda}\|(\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\|_2^2, \tag{29}$$

$$\boldsymbol{R}_{t+1} = \underset{\boldsymbol{R}\in\mathbb{R}^{r\times n}}{\operatorname{argmin}} \; F(\boldsymbol{L}_t\boldsymbol{R}_t) + \langle \nabla_{\boldsymbol{R}} F(\boldsymbol{L}_t\boldsymbol{R}_t), \boldsymbol{R} - \boldsymbol{R}_t\rangle + \frac{1}{2\lambda}\|\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t)\|_2^2. \tag{30}$$

is given by

$$v \tag{31}$$

Proof. Let $\Delta \boldsymbol{L} := \boldsymbol{L} - \boldsymbol{L}_\top$, $\boldsymbol{G} = \nabla_{\boldsymbol{L}} F(\boldsymbol{L}_t \boldsymbol{R}_t)$, and let $\boldsymbol{R}_t = \boldsymbol{U}_R \boldsymbol{\Sigma}_R \boldsymbol{V}_R^\top$ Now, reparameterize $\Delta \boldsymbol{L} = \boldsymbol{X} \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top$. Rewriting (29),

$$\begin{aligned}
\min_{\boldsymbol{L} \in \mathbb{R}^{m \times r}} \langle \boldsymbol{G}, \Delta \boldsymbol{L} \rangle + \frac{1}{2\lambda} \|\Delta \boldsymbol{L} \boldsymbol{R}_t\|_2^2 &= \min_{\boldsymbol{X} \in \mathbb{R}^{m \times r}} \left\langle \boldsymbol{G}, \boldsymbol{X} \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top \right\rangle + \frac{1}{2\lambda} \|\boldsymbol{X} \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top \boldsymbol{R}_t\|_2^2 \\
&= \min_{\boldsymbol{X} \in \mathbb{R}^{m \times r}} \left\langle \boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1}, \boldsymbol{X} \right\rangle + \frac{1}{2\lambda} \|\boldsymbol{X} \boldsymbol{V}_R^\top\|_2^2 \\
&= \min_{\boldsymbol{X} \in \mathbb{R}^{m \times r}} \left\langle \boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1}, \boldsymbol{X} \right\rangle + \frac{1}{2\lambda} \|\boldsymbol{X}\|_2^2
\end{aligned}$$

The minimizer is simply $\boldsymbol{X} = \text{polar}(\boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1})$, from which

$$\begin{aligned}
\Delta \boldsymbol{L} = \text{polar}\left(\boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1}\right) \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top &= \text{polar}\left(\boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1}\right) \boldsymbol{V}_R^\top \boldsymbol{V}_R \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top \\
&= \text{polar}\left(\boldsymbol{G} \boldsymbol{U}_R \boldsymbol{\Sigma}_R^{-1} \boldsymbol{V}_R^\top\right) \boldsymbol{V}_R \boldsymbol{\Sigma}_R^{-1} \boldsymbol{U}_R^\top \\
&= \text{polar}\left(\boldsymbol{G} {\boldsymbol{R}_t}^{+\top}\right) {\boldsymbol{R}_t}^+
\end{aligned}$$

Now recall that $\boldsymbol{G} = \widehat{\boldsymbol{G}} \boldsymbol{R}_\top$, where $\widehat{\boldsymbol{G}} = \nabla_{\boldsymbol{L}_t} F(\boldsymbol{L}_t)$ evaluated at $\boldsymbol{L}_t = \boldsymbol{L}_t \boldsymbol{R}_t$. This gives

$$\boldsymbol{G} {\boldsymbol{R}_t}^{+\top} = \widehat{\boldsymbol{G}} \boldsymbol{V}_R \boldsymbol{V}_R^\top = \widehat{\boldsymbol{G}} \boldsymbol{V}_R \boldsymbol{U}_R^\top \boldsymbol{U}_R \boldsymbol{V}_R^\top = \widehat{\boldsymbol{G}} \, \text{polar} \boldsymbol{R}_t^\top \text{polar}(\boldsymbol{R}_t)$$

Substituting back,

$$\begin{aligned}
\Delta \boldsymbol{L} &= \text{polar}\left(\widehat{\boldsymbol{G}} \, \text{polar} \boldsymbol{R}_t^\top \text{polar}(\boldsymbol{R}_t)\right) {\boldsymbol{R}_t}^+ \\
&= \text{polar}\left(\widehat{\boldsymbol{G}} \, \text{polar} \boldsymbol{R}_t^\top\right) \text{polar}(\boldsymbol{R}_t) {\boldsymbol{R}_t}^+ \\
&= \text{polar}\left(\widehat{\boldsymbol{G}} \, \text{polar} \boldsymbol{R}_t^\top\right) \left(\text{polar}(\boldsymbol{R}_t) \cdot {\boldsymbol{R}_t}^\top\right)^{-1}
\end{aligned}$$

We can use PolarExpress to compute $\text{polar}(\boldsymbol{R}_t)$, then use Newton-Schulz to compute the inverse of the the $r \times r$ psd matrix $\left(\text{polar}(\boldsymbol{R}_t) \cdot {\boldsymbol{R}_t}^\top\right)$.

Notice also that $\Delta \boldsymbol{L} \cdot \boldsymbol{R}_t = \text{polar}\left(\widehat{\boldsymbol{G}} \, \text{polar} \boldsymbol{R}_t^\top\right) \text{polar}(\boldsymbol{R}_t) = \text{polar}(\widehat{\boldsymbol{G}}) \boldsymbol{V}_R \boldsymbol{V}_R^\top$, I think? By the same argument, we can solve for $\Delta \boldsymbol{R} = \left(\text{polar} \boldsymbol{L}_t^\top \cdot \boldsymbol{L}_t\right)^{-1} \text{polar}\left(\text{polar} \boldsymbol{L}_t^\top \, \widehat{\boldsymbol{G}}\right)$ (I think?) and $\boldsymbol{L} \cdot \Delta \boldsymbol{R} = \text{polar}(\boldsymbol{L}_t) \text{polar}\left(\text{polar} \boldsymbol{L}_t^\top \widehat{\boldsymbol{G}}\right)$. For simplicity, let's now impose that $\boldsymbol{R}_t$ and $\boldsymbol{R}$ are always semi-orthogonal, so that $\boldsymbol{\Sigma}_R = \boldsymbol{I}$.

But Is there an iterative method that gives it to us all at once? Is there a simple polynomial in $\widehat{\boldsymbol{G}}$ and $\boldsymbol{R}_t$ to which this is the fixed point?

Maybe we should think of the end goal. We previously had $\boldsymbol{LR}$. We want to move that to $\boldsymbol{LR} + \lambda \text{polar}(\widehat{\boldsymbol{G}})$. And then, we want a low rank approximation to that. All this without actually computing $\text{polar}(\widehat{\boldsymbol{G}})$, which would be too expensive probably. Maybe better to keep it factored in SVD form. $\boldsymbol{LDR}$. Then assuming $\boldsymbol{LDR} + \lambda \text{polar}(\widehat{\boldsymbol{G}}) \approx \boldsymbol{LDR}$, we right multiply by $\boldsymbol{R}_\top \boldsymbol{D}^{-1}$ and then do a few PolarExpress iterations to get that back to orthogonal. Then maybe use that to

hit the thing again. Now we have an estimate for a new $\boldsymbol{L}$. Still, you have to compute polar($\widehat{\boldsymbol{G}}$). Perhaps we could switch the order, using polar($\widehat{\boldsymbol{G}}$)$\boldsymbol{R}_\top \approx$ polar($\widehat{\boldsymbol{G}}\boldsymbol{R}_\top$). Can we prove that this converges to the same thing?

□

5 Momentum and Adam with product norm

Consider the following preconditioner momentum method

$$\boldsymbol{M}^{t+1} = \beta \boldsymbol{M}^t + (1-\beta)\boldsymbol{G}_t \tag{32}$$
$$\boldsymbol{W}^{t+1} = \boldsymbol{W}^t - \gamma_t \boldsymbol{D}_t^{\odot -1} \odot \boldsymbol{M}^{t+1} \tag{33}$$

where $\gamma_t > 0$ is the learning rate, $\boldsymbol{D}$ a preconditioner and $\boldsymbol{M}_t$ a momentum estimate of the gradient. Here we use $\boldsymbol{D}^{\odot -1}$ to denote the entry-wise inverse of $\boldsymbol{D}$, and $\odot$ to be the entry-wise (Hadamard) product.

We can adapt this method to make use of this product norm by using the following variational viewpoint (32)–(33)

$$\boldsymbol{W}^{t+1} = \operatorname*{argmin}_{\boldsymbol{W}} \left\langle \boldsymbol{M}^{t+1}, \boldsymbol{W} - \boldsymbol{W}^t \right\rangle + \frac{1}{2\gamma_t} \|\boldsymbol{D}_t^{1/2} \odot (\boldsymbol{W} - \boldsymbol{W}^t)\|_F^2, \tag{34}$$

where $\langle \boldsymbol{M}, \boldsymbol{W} \rangle = \operatorname{tr}\left(\boldsymbol{M}^\top \boldsymbol{W}\right)$. Using a coordinate change $\hat{\boldsymbol{W}} := \boldsymbol{D}_t^{1/2} \odot \boldsymbol{W}$, the above is equivalent to

$$\hat{\boldsymbol{W}}^{t+1} = \operatorname*{argmin}_{\boldsymbol{W}} \left\langle \boldsymbol{M}^{t+1}, \boldsymbol{D}_t^{-1/2} \odot (\hat{\boldsymbol{W}} - \hat{\boldsymbol{W}}^t) \right\rangle + \frac{1}{2\gamma_t} \|\hat{\boldsymbol{W}} - \hat{\boldsymbol{W}}^t\|_F^2 \tag{35}$$
$$\boldsymbol{W}^{t+1} = \boldsymbol{D}_t^{-1/2} \odot \hat{\boldsymbol{W}}^{t+1} \tag{36}$$

Now suppose the parameters can be grouped such that $\boldsymbol{W} = (\boldsymbol{L}, \boldsymbol{R})$, and that we know only the product $\boldsymbol{LR}$ appears in the loss. Let $\boldsymbol{D}_{\boldsymbol{L}}$ and $\boldsymbol{M}_{\boldsymbol{L}}$ be the entries that correspond to $\boldsymbol{L}$, and analogously for $\boldsymbol{D}_{\boldsymbol{R}}$ and $\boldsymbol{M}_{\boldsymbol{R}}$. Following the reasoning of the previous section, we should change the regularizor to take account of the product. For instance, the update for $\boldsymbol{L}$ in (25) with momentum and preconditioning would be given by

$$\begin{aligned}
\boldsymbol{M}_{\boldsymbol{L}}^{t+1} &= \beta \boldsymbol{M}_{\boldsymbol{L}}^t + (1-\beta)\nabla_{\boldsymbol{L}} F(\boldsymbol{\Delta W}^t) \\
\boldsymbol{L}_{t+1} &= \boldsymbol{D}_{\boldsymbol{L}}^{-1/2} \odot \operatorname*{argmin}_{\boldsymbol{L} \in \mathbb{R}^{m\times r}} \left\langle \boldsymbol{M}_{\boldsymbol{L}}^{t+1}, \boldsymbol{D}_{\boldsymbol{L}}^{-1/2} \odot (\boldsymbol{L} - \boldsymbol{L}_t) \right\rangle + \frac{1}{2\lambda} \|(\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\|_F^2. \qquad (37) \\
\boldsymbol{M}_{\boldsymbol{R}}^{t+1} &= \beta \boldsymbol{M}_{\boldsymbol{R}}^t + (1-\beta)\nabla_{\boldsymbol{R}} F(\boldsymbol{\Delta W}^t) \\
\boldsymbol{R}_{t+1} &= \boldsymbol{D}_{\boldsymbol{R}}^{-1/2} \odot \operatorname*{argmin}_{\boldsymbol{R} \in \mathbb{R}^{r\times n}} \left\langle \boldsymbol{M}_{\boldsymbol{R}}^{t+1}, \boldsymbol{D}_{\boldsymbol{R}}^{-1/2} \odot (\boldsymbol{R} - \boldsymbol{R}_t) \right\rangle + \frac{1}{2\lambda} \|\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t)\|_F^2. \qquad (38)
\end{aligned}$$

Lemma 5.1. The solution to (37)–(38) is given by

$$\boldsymbol{L} = \boldsymbol{L}_t - \lambda(\boldsymbol{D}_{\boldsymbol{L}}^{-1} \odot \boldsymbol{M}_{\boldsymbol{L}}^{t+1})(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger. \tag{39}$$

$$\boldsymbol{R} = \boldsymbol{R}_t - \lambda(\boldsymbol{L}_t^\top\boldsymbol{L}_t)^\dagger(\boldsymbol{D}_{\boldsymbol{R}}^{-1} \odot \boldsymbol{M}_{\boldsymbol{R}}^{t+1}). \tag{40}$$

In words, the resulting updates (39) and (40) are equivalent to applying (31) and (28), but with the gradients replaced by the preconditioned momentum estimates.

Proof. Using that

$$\nabla_{\boldsymbol{C}} \langle \boldsymbol{L}_t, \boldsymbol{R}_t \odot \boldsymbol{C}\rangle = \boldsymbol{R}_t \odot \boldsymbol{L}_t,$$

setting the gradient of (37) and (38) to zero gives

$$\lambda \boldsymbol{D}_{\boldsymbol{L}}^{-1/2}\boldsymbol{M}_{\boldsymbol{L}}^{t+1} + (\boldsymbol{L} - \boldsymbol{L}_t)\boldsymbol{R}_t\boldsymbol{R}_t^\top = 0.$$
$$\lambda \boldsymbol{D}_{\boldsymbol{R}}^{-1/2}\boldsymbol{M}_{\boldsymbol{R}}^{t+1} + (\boldsymbol{L}_t)^\top\boldsymbol{L}_t(\boldsymbol{R} - \boldsymbol{R}_t) = 0.$$

Solving for $\boldsymbol{L}$ and $\boldsymbol{R}$ gives

$$\boldsymbol{L} = \boldsymbol{L}_t - \gamma \boldsymbol{D}_{\boldsymbol{L}}^{-1/2}\boldsymbol{M}_{\boldsymbol{L}}^{t+1}(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger$$
$$\boldsymbol{R} = \boldsymbol{R}_t - \gamma(\boldsymbol{L}_t^\top\boldsymbol{L}_t)^\dagger\boldsymbol{D}_{\boldsymbol{L}}^{-1/2}\boldsymbol{M}_{\boldsymbol{L}}^{t+1}.$$

□

Left multiplying and right multiplying by $\boldsymbol{D}_{\boldsymbol{L}}^{-1/2}$ $\boldsymbol{D}_{\boldsymbol{R}}^{-1/2}$ the first and second equation, respectively gives (39).

Contrast this to the Adam variant given in [**zhang2024riemannianpreconditionedlorafinetuning**], which instead applies the standard Adam update using the preconditioned gradients $\boldsymbol{D}_{\boldsymbol{L}}^{-1/2} \odot \nabla_{\boldsymbol{L}} F(\boldsymbol{\Delta W}^t)$ and $\boldsymbol{D}_{\boldsymbol{R}}^{-1/2} \odot \nabla_{\boldsymbol{R}} F(\boldsymbol{\Delta W}^t)$. For instance, for the $\boldsymbol{L}$ component they use the method

$$\begin{aligned}\boldsymbol{M}_{\boldsymbol{L}}^{t+1} &= \beta \boldsymbol{M}_{\boldsymbol{L}}^t + (1-\beta)\nabla_{\boldsymbol{L}} F(\boldsymbol{\Delta W}^t)(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger \\ \boldsymbol{L}_{t+1} &= \boldsymbol{L}_t - \gamma \boldsymbol{D}_{\boldsymbol{L}}^{-1}\boldsymbol{M}_{\boldsymbol{L}}^{t+1},\end{aligned} \tag{41}$$

where the preconditioner $\boldsymbol{D}_{\boldsymbol{L}}^{-1}$ would also be computed using $\nabla_{\boldsymbol{L}} F(\boldsymbol{\Delta W}^t)(\boldsymbol{R}_t\boldsymbol{R}_t^\top)^\dagger$ as opposed to the gradient.

Robert: It's not clear to me which approach makes more sense or would be better. In our approach in Lemma 5.1, we apply the product preconditioning to the Adam direction. In (41) from [**zhang2024riemannianpreconditionedlorafinetuning**], they apply the Adam method to the product preconditioned gradient. Both are reasonable. But given the wide spread use of Lora, it would be worth comparing them, in case our new approach works better.

6 A toy problem

Consider the least squares problem

$$\min_{\boldsymbol{W}} \|\boldsymbol{X}_{\text{init}}\boldsymbol{W} - \boldsymbol{Y}_{\text{init}}\|_F^2.$$

This is a least squares problem, with optimal solution $\boldsymbol{W}^\star = \boldsymbol{X}_{\text{init}}^\dagger \boldsymbol{Y}_{\text{init}}$. Let $\boldsymbol{X} = \boldsymbol{X}_{\text{init}} + \boldsymbol{E}_{\boldsymbol{X}}$ and $\boldsymbol{Y} = \boldsymbol{Y}_{\text{init}} + \boldsymbol{E}_{\boldsymbol{Y}}$. Now consider the fine-tuning problem

$$\min_{\boldsymbol{L},\boldsymbol{R}} \|\boldsymbol{X}(\boldsymbol{W}^\star + \boldsymbol{L}\boldsymbol{R}) - \boldsymbol{Y}\|_F^2 = \|\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - (\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{W}^\star)\|_F^2 = \|\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - \boldsymbol{Z}\|_F^2,$$

where $\boldsymbol{Z} = \boldsymbol{Y} - \boldsymbol{X}\boldsymbol{W}^\star$. This is a constrained low-rank approximation problem, whose optimal objective value is given by

$$\|(\boldsymbol{I} - \boldsymbol{P}_{\boldsymbol{X}})\boldsymbol{Z}\|_F^2 + \sum_{i>r} \sigma_i(\boldsymbol{P}_{\boldsymbol{X}}\boldsymbol{Z})^2 = \|(\boldsymbol{I} - \boldsymbol{P}_{\boldsymbol{X}})\boldsymbol{Y}\|_F^2 + \sum_{i>r} \sigma_i(\boldsymbol{P}_{\boldsymbol{X}}\boldsymbol{Y} - \boldsymbol{X}\boldsymbol{W}^\star)^2,$$

where $\boldsymbol{P}_{\boldsymbol{X}}$ denotes the orthogonal projection onto range($\boldsymbol{X}$).

6.1 The gradient of the loss

Consider the function

$$F(\Delta\boldsymbol{W}) = \|\boldsymbol{X}(\boldsymbol{W}^\star + \boldsymbol{\Delta}\boldsymbol{W}) - \boldsymbol{Y}\|_F^2 = \|\boldsymbol{X}\boldsymbol{\Delta}\boldsymbol{W} - \boldsymbol{Z}\|_F^2.$$

The gradient is given by

$$\begin{aligned}\nabla F(\boldsymbol{\Delta}\boldsymbol{W}) &= 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Delta}\boldsymbol{W} - 2\boldsymbol{X}^T\boldsymbol{Z} = 2\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\Delta}\boldsymbol{W} - \boldsymbol{Z}) \\ &= 2\boldsymbol{X}^T(\boldsymbol{X}(\boldsymbol{W}^\star + \boldsymbol{\Delta}\boldsymbol{W}) - \boldsymbol{Y}).\end{aligned}$$

Hence, PGD takes the form

$$\boldsymbol{\Delta}\boldsymbol{W}_{t+1} = \mathcal{T}_r(\boldsymbol{\Delta}\boldsymbol{W}_t - 2\lambda\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\Delta}\boldsymbol{W}_t - \boldsymbol{Z}))$$

6.2 Gradients of low-rank factors

Consider the function

$$G(\boldsymbol{L}, \boldsymbol{R}) = \|\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - \boldsymbol{Z}\|_F^2$$

Then,

$$\begin{aligned}\nabla_{\boldsymbol{L}} G(\boldsymbol{L}, \boldsymbol{R}) &= 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{L}\boldsymbol{R}\boldsymbol{R}^T - 2\boldsymbol{X}^T\boldsymbol{Z}\boldsymbol{R}^T, \\ \nabla_{\boldsymbol{R}} G(\boldsymbol{L}, \boldsymbol{R}) &= 2\boldsymbol{L}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - 2\boldsymbol{L}^T\boldsymbol{X}^T\boldsymbol{Z}.\end{aligned}$$

Hence, the update rule is

$$\begin{bmatrix}\boldsymbol{L}_{t+1} \\ \boldsymbol{R}_{t+1}\end{bmatrix} = \begin{bmatrix}\boldsymbol{L}_t - 2\lambda\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - \boldsymbol{Z})\boldsymbol{R}^T \\ \boldsymbol{R}_t - 2\lambda\boldsymbol{L}^T\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{L}\boldsymbol{R} - \boldsymbol{Z})\end{bmatrix}$$

7 Templates

Theorem 7.1. I like my theorems like this, with boldface vectors $\mathbf{x}$ and matrices $\boldsymbol{L}_t$

Algorithm 5: `BEST`: Better Estimator of Subtantial Truth.

Default settings: $\alpha_k = 1$
Input: $\mathbf{x}_0 \in \mathbb{R}^d$
Init: $\bar{f}_0 = f(\mathbf{x}_0)$ **for** $k = 1$ ***to*** $K - 1$ **do**
    Improve something ;
    Store something ;
    report something ;
    return The best $\mathbf{x}$

Or

Algorithm 6: `BEST`: Better Estimator of Substantial Truth.

Default settings: $\mathbf{x}_0 = 0 \in \mathbb{R}^d, h_0 = 0$
Input: $\mathbf{x}_0 \in \mathbb{R}^d$, $h_0 > 0$
for $k = 1$ ***to*** $K - 1$ **do**
    $\mathbf{x}_{n+1} = \mathbf{x}_n - h_n \nabla f(\mathbf{x}_n)$
    $\mathbf{x}_{n+1}^H = \mathbf{x}_n - \frac{1}{2} h_n (\nabla f(\mathbf{x}_n) + \nabla f(\mathbf{x}_{n+1}))$
Output: The best $\mathbf{x}$

It's also nice to use clever referencing like algorithm 6 or Algorithm 5.